



UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
INGENIERIA EN SISTEMAS DE INFORMACION Y CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software
Ing. Edgar Raúl Molina Rey

Proyecto Final Automatización de procesos con n8n

EDDY ADÁN BOTZOC MERIDA 4090-22-14645

COBÁN ALTA VERAPAZ, Mayo 2026

SISTEMA DE CONTROL DE INVENTARIO CON ALERTAS PREDICTIVAS SIMPLES UTILIZANDO N8N Y FLASK

Contenido

Introducción	2
Planteamiento del problema.....	3
Objetivos	3
Objetivo general	3
Objetivos específicos	3
Alcance del sistema.....	4
Arquitectura del sistema.....	4
Tecnologías utilizadas.....	5
Modelo de datos	5
Productos.....	5
Ventas	5
Alertas.....	6
Auditoria	6
Descripción de workflows	6
WF1 – Registro de ventas	6
WF2 – Procesamiento de stock y alertas	6
WF3 – Reportes automáticos	7
Manejo de errores y bitácora	7
Seguridad	7
Dashboard y visualización	8
Evidencia de pruebas.....	8
Conclusiones.....	13

Introducción

En la actualidad, muchas pequeñas y medianas empresas presentan dificultades en el control manual de inventarios, provocando pérdidas económicas, errores en el stock y retrasos en la

reposición de productos.

Para solucionar esta problemática, se desarrolló un sistema automatizado de gestión de inventario utilizando n8n, Flask y PostgreSQL en un entorno completamente local mediante Docker.

El sistema permite registrar ventas, actualizar automáticamente el stock de productos, generar alertas de bajo inventario, visualizar reportes y mantener una auditoría de las operaciones realizadas.

Planteamiento del problema

Las empresas que gestionan inventarios manualmente enfrentan problemas como:

- Descontrol del stock disponible.
- Errores humanos en registros de ventas.
- Falta de alertas ante productos críticos.
- Poca trazabilidad de movimientos.
- Retrasos en reabastecimiento.

Debido a esto, surge la necesidad de implementar un sistema automatizado que permita controlar el inventario en tiempo real y mejorar la gestión de productos.

Objetivos

Objetivo general

Desarrollar un sistema automatizado de control de inventario utilizando n8n y Flask que permita gestionar productos, ventas y alertas de stock bajo en un entorno local.

Objetivos específicos

- Registrar ventas automáticamente.
- Actualizar el stock en tiempo real.
- Generar alertas cuando un producto tenga bajo stock.
- Visualizar reportes y estadísticas.
- Mantener una auditoría de operaciones.
- Gestionar usuarios mediante roles.

Alcance del sistema

El sistema permite:

- Gestión de productos.
- Registro de ventas.
- Control de inventario.
- Generación de alertas.
- Dashboard administrativo.
- Auditoría de eventos.
- Reportes exportables.
- Gestión de usuarios y roles.

El proyecto funciona completamente en entorno local mediante Docker y n8n.

Arquitectura del sistema

El sistema está compuesto por:

- Frontend web desarrollado con HTML, CSS y Bootstrap.
- Backend desarrollado con Flask en Python.
- Automatización de procesos utilizando n8n.
- Base de datos PostgreSQL.
- Contenedores Docker para despliegue local.

Flujo general

1. El usuario registra una venta desde Flask.
2. Flask envía la solicitud al webhook de n8n.
3. n8n procesa la venta.
4. PostgreSQL actualiza el stock.
5. Si existe bajo stock, se genera una alerta.
6. El sistema registra auditoría y notificaciones.

Tecnologías utilizadas

Tecnología	Uso
Python Flask	Backend
HTML/CSS/Bootstrap	Frontend
PostgreSQL	Base de datos
n8n	Automatización
Docker	Contenedores
Chart.js	Gráficas
JavaScript	Interactividad

Modelo de datos

El sistema utiliza las siguientes tablas principales:

Productos

Campo	Tipo
id	SERIAL
sku	VARCHAR
nombre	VARCHAR
stock_actual	INTEGER
stock_minimo	INTEGER

Ventas

Campo	Tipo
id	SERIAL
sku	VARCHAR
cantidad	INTEGER

fecha	TIMESTAMP
origen	VARCHAR

Alertas

Campo	Tipo
id	SERIAL
sku	VARCHAR
estado	VARCHAR
fecha	TIMESTAMP

Auditoria

Campo	Tipo
id	SERIAL
fecha	TIMESTAMP
evento	VARCHAR
detalle	TEXT

Descripción de workflows

WF1 – Registro de ventas

Este workflow recibe solicitudes desde Flask mediante un webhook y registra las ventas realizadas por los usuarios.

Proceso:

1. Recepción del webhook.
2. Validación de datos.
3. Registro de venta.
4. Respuesta al sistema web.

WF2 – Procesamiento de stock y alertas

Workflow encargado de actualizar el stock y generar alertas de bajo inventario.

Proceso:

1. Consultar producto.
2. Validar existencia.
3. Actualizar stock.
4. Verificar bajo stock.
5. Crear alerta.
6. Registrar auditoría.
7. Enviar notificación.

WF3 – Reportes automáticos

Genera reportes exportables y consultas estadísticas.

Funciones:

- Reportes de ventas.
- Productos críticos.
- Alertas pendientes.
- Exportación a Excel.

Manejo de errores y bitácora

El sistema implementa:

- Validaciones de stock.
- Verificación de productos inexistentes.
- Prevención de alertas duplicadas.
- Registro de auditoría.
- Control de errores en workflows.

Además, se mantiene una bitácora de eventos importantes del sistema.

Seguridad

El sistema implementa:

- Inicio de sesión.

- Roles de usuario.
- Restricción de módulos.
- Validación de formularios.
- Protección de rutas mediante sesiones.

Roles implementados:

- Administrador
- Inventario
- Ventas
- Supervisor

Dashboard y visualización

El dashboard incluye:

- Total de productos.
- Total de ventas.
- Productos críticos.
- Alertas pendientes.
- Ventas por producto.
- Stock actual.
- Ventas por día.

Las gráficas fueron implementadas con Chart.js.

Evidencia de pruebas

Se realizaron pruebas de:

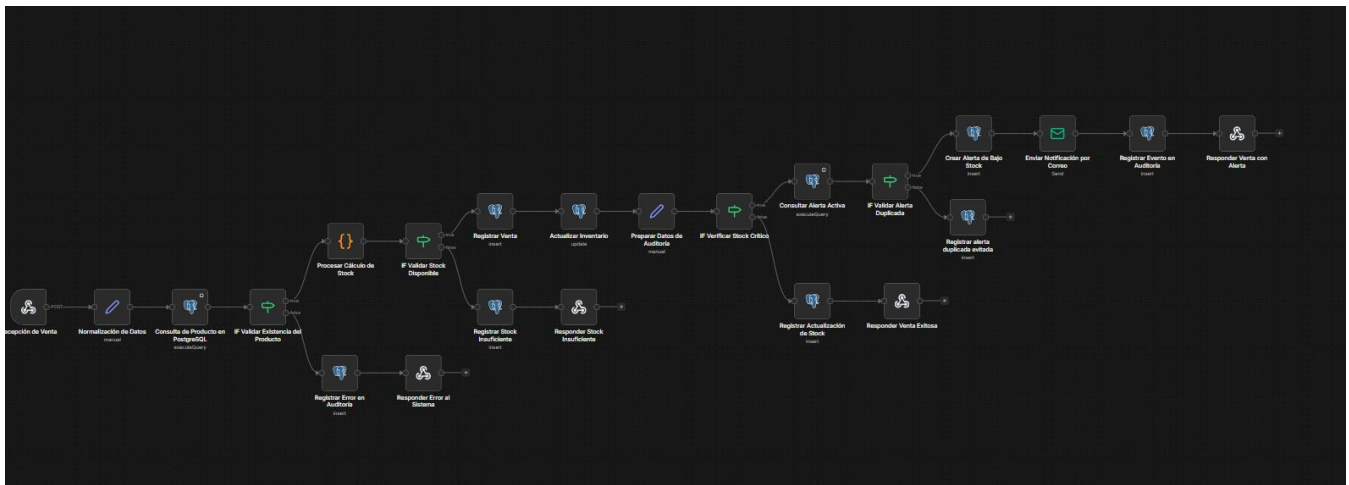
- Registro de productos.
- Registro de ventas.
- Actualización automática de stock.
- Generación de alertas.
- Exportación de reportes.
- Gestión de usuarios.

- Dashboard dinámico.

Resultados:

- El sistema actualiza el stock correctamente.
- Las alertas se generan automáticamente.
- La auditoría registra eventos importantes.
- Los reportes se exportan correctamente.

Diagrama del workflow principal n8n



Explicación del Workflow Principal (WF2) – Procesamiento de Stock y Alertas

1. Recepción de Venta (Webhook)

El workflow inicia mediante un Webhook que recibe la información enviada desde la aplicación web, incluyendo el SKU del producto y la cantidad vendida.

2. Normalización de Datos

Se procesan y organizan los datos recibidos para garantizar que el formato sea correcto antes de continuar con el flujo.

3. Consulta de Producto en PostgreSQL

El sistema realiza una consulta a la base de datos PostgreSQL para verificar si el producto existe y obtener su información actual, como nombre, stock disponible y stock mínimo.

4. Validar Existencia del Producto

Se valida si el producto consultado realmente existe en la base de datos.

- Si el producto no existe:
 - Se registra el error en auditoría.
 - Se envía una respuesta de error al sistema web.
- Si el producto existe:
 - El flujo continúa con el procesamiento de la venta.

5. Procesar Cálculo de Stock

Mediante un nodo JavaScript se calcula el nuevo stock del producto restando la cantidad vendida al stock actual.

6. Validar Stock Disponible

Se verifica si el producto tiene suficiente inventario para completar la venta.

- Si no hay stock suficiente:
 - Se registra el evento de stock insuficiente.
 - Se responde al sistema indicando que la venta no puede realizarse.
- Si el stock es suficiente:
 - El flujo continúa normalmente.

7. Registrar Venta

La venta realizada se almacena en la tabla de ventas de PostgreSQL, registrando el SKU, cantidad vendida, fecha y origen de la operación.

8. Actualizar Inventario

Se actualiza el stock del producto en la base de datos con el nuevo valor calculado.

9. Preparar Datos de Auditoría

El sistema prepara la información necesaria para registrar los eventos en la tabla de auditoría.

10. Verificar Stock Crítico

Se valida si el nuevo stock del producto quedó por debajo o igual al stock mínimo permitido.

- Si el stock es normal:
 - Se registra únicamente la actualización de stock.
 - Se responde exitosamente al sistema web.
- Si el stock es crítico:
 - El flujo continúa hacia la generación de alertas.

11. Consultar Alerta Activa

El sistema consulta si ya existe una alerta activa para el producto con bajo stock.

12. Validar Alerta Duplicada

Se evita generar múltiples alertas repetidas para un mismo producto.

- Si ya existe una alerta activa:
 - Se registra que la alerta duplicada fue evitada.
- Si no existe una alerta:
 - Se crea una nueva alerta.

13. Crear Alerta de Bajo Stock

Se registra una alerta pendiente en la base de datos indicando que el producto alcanzó un nivel crítico de inventario.

14. Enviar Notificación por Correo

El sistema envía automáticamente una notificación por correo electrónico indicando el producto afectado y su nivel de stock.

15. Registrar Evento en Auditoría

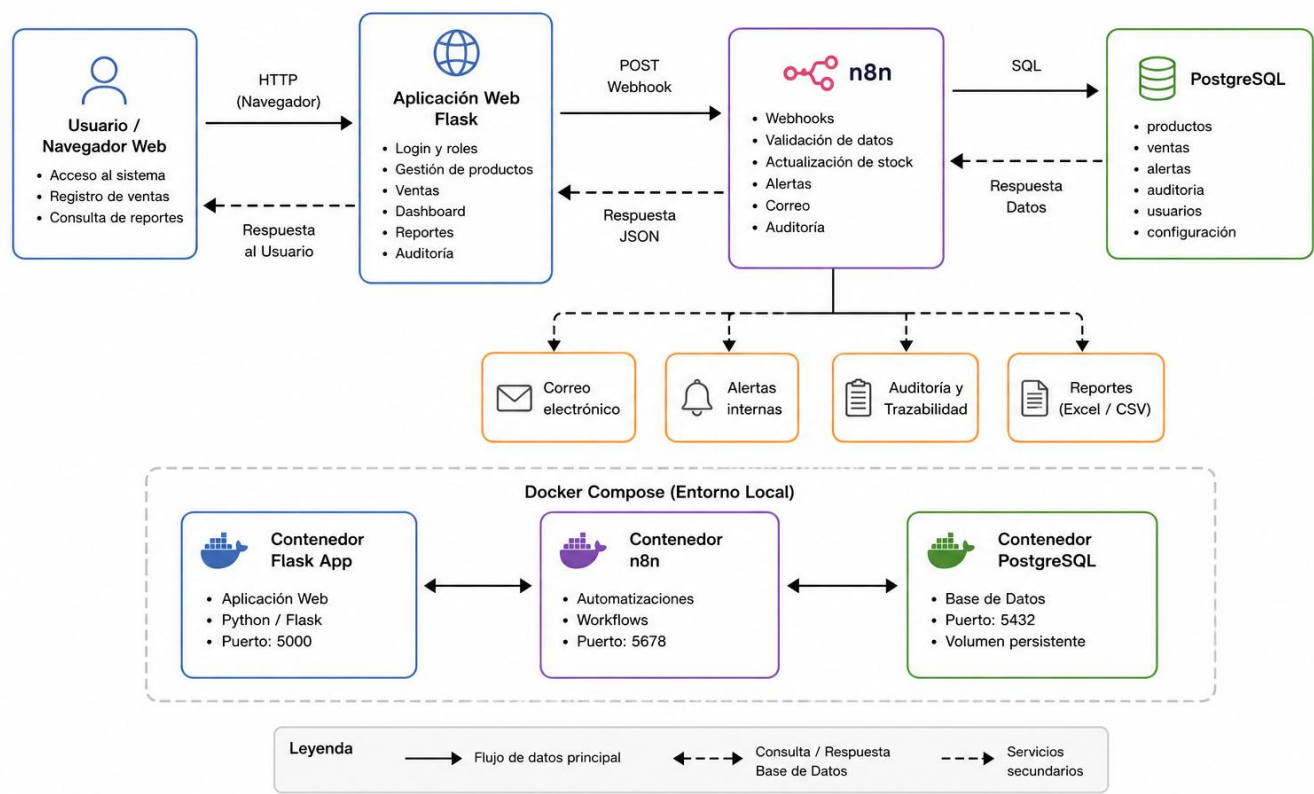
Todos los eventos importantes del proceso quedan registrados en la tabla de auditoría para mantener trazabilidad y control del sistema.

16. Responder Venta con Alerta

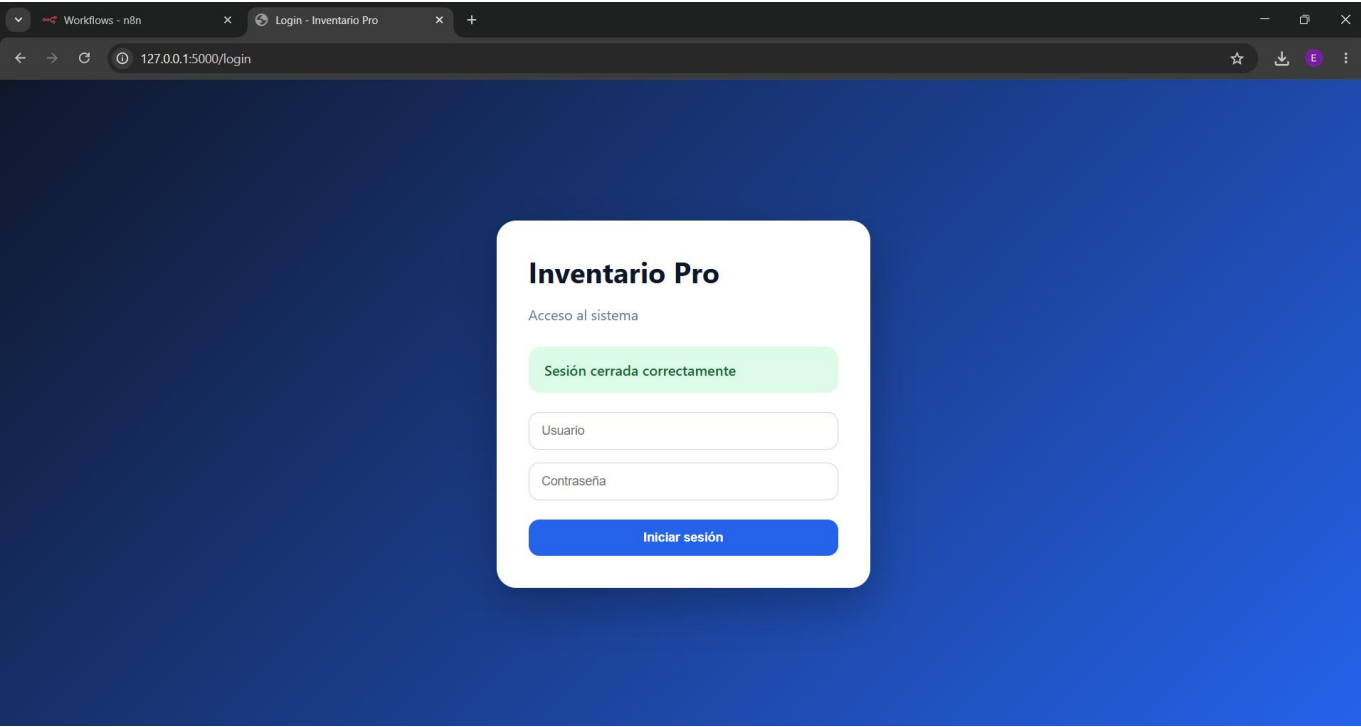
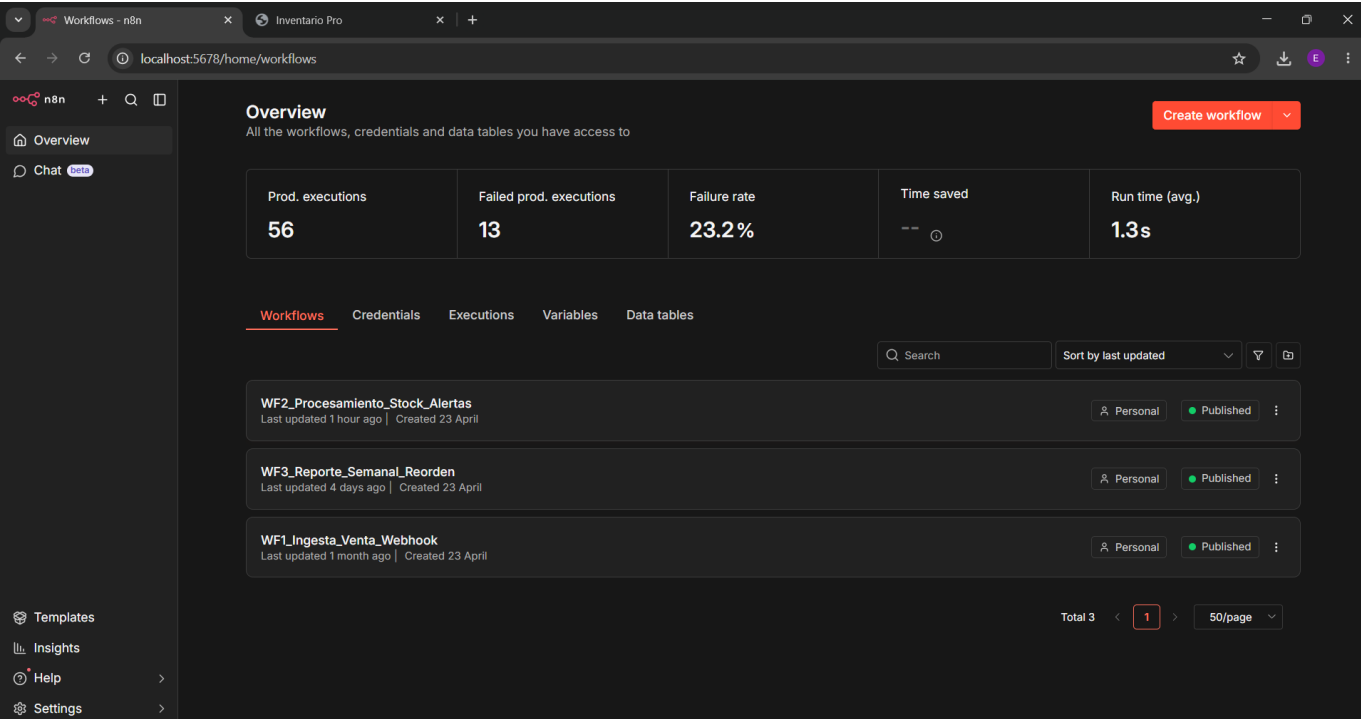
Finalmente, el workflow responde exitosamente a la aplicación web indicando que la venta fue procesada y que el producto quedó en estado de bajo stock.

Diagrama de arquitectura (ej. contenedores y flujo)

Diagrama de Arquitectura del Sistema de Gestión de Inventario



Capturas de pantalla



Workflows - n8n

Inventario Pro

127.0.0.1:5000/dashboard

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoría

Sistema de Gestión de Inventario

Total de productos

27

Total de ventas

88

Ventas de hoy

0

Alertas pendientes

1

Productos críticos

1

Ventas por producto

SKU	PRODUCTO	VENDIDO
Azucar 1lb	266	
Agua embotellada	255	
Arroz 1lb	54	
Carton de huevos	31	
Aceite vegetal 1L	25	
Cafe 500g	74	

Stock actual por producto

SKU	PRODUCTO	STOCK
Leche Liquida 1L	2	9
Carton de huevos	3	5
Frijol en bola 1 Lbr	9	3
Queso fresco 1lb	12	3
Shampoo 400ml	15	4
Arroz 4lb	45	1

Workflows - n8n

Inventario Pro

127.0.0.1:5000/productos

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoría

Sistema de Gestión de Inventario

Gestión de Productos

+ Nuevo producto

Buscar por SKU o nombre

Buscar

Limpiar

ID	SKU	NOMBRE	STOCK ACTUAL	STOCK MÍNIMO	ÚLTIMA ACTUALIZACIÓN	ACCIONES
32	SALCHICHAS-024	Salchichas de pavo ahumadas	25	8	2026-05-30 23:55:36.896077	Editar Eliminar
31	GASEOSA-023	Pepsi	35	8	2026-05-30 23:36:34.117446	Editar Eliminar
30	TEQUILA-022	Tequila Don Julio	25	5	2026-05-30 06:07:01.945907	Editar Eliminar
29	CREMA-021	Medio litro de crema	25	8	2026-05-29 21:50:01.065570	Editar Eliminar
28	QUESO-020	Queso fresco 1lb	12	3	2026-05-29 21:49:26.301846	Editar Eliminar
27	MANTEQUILLA-021	Mantequilla 250g	16	4	2026-05-29 21:49:11.152056	Editar Eliminar

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoría

Sistema de Gestión de Inventario

Registrar venta

Seleccione producto

Cantidad

Procesar venta

Últimas ventas

Buscar por SKU

dd/mm/aaaa

Filtrar

Limpiar

ID	FECHA	SKU	CANTIDAD	ORIGEN
89	2026-05-30 20:05:09.648000	HUEVOS-012	15	web
88	2026-05-30 20:02:40.575000	ACEITE-009	1	web
87	2026-05-30 19:58:59.177000	AZUCAR-002	1	web
86	2026-05-30 19:58:47.523000	AZUCAR-002	20	web

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoría

Sistema de Gestión de Inventario

Alertas del sistema

Buscar SKU, tipo o mensaj

Todos los estados

Todas las prioridades

Filtrar

Limpiar

ID	FECHA	SKU	TIPO	MENSAJE	PRIORIDAD	ESTADO	RESOLUCIÓN	ACCIONES
35	2026-05-30 20:05:09.693000	HUEVOS-012	BAJO_STOCK	Stock bajo en Carton de huevos. Stock actual: 3, Stock mínimo: 6	MEDIA	PENDIENTE	Sin resolver	En proceso Atendida Reabastecida
34	2026-05-30 19:58:47.556000	AZUCAR-002	BAJO_STOCK	Stock bajo en Azucar 1lb. Stock actual: 1, Stock mínimo: 3	MEDIA	REABASTECIDA	2026-05-31 00:00:08.100518 Usuario: admin	Cerrada
33	2026-05-30 19:57:14.304000	AGUA-003	BAJO_STOCK	Stock bajo en Agua embotellada. Stock actual: 3, Stock mínimo: 5	MEDIA	REABASTECIDA	2026-05-30 23:57:59.773815 Usuario: admin	Cerrada

Workflows - n8n

Inventario Pro

127.0.0.1:5000/reportes

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoría

Sistema de Gestión de Inventario

Reportes descargables

Generar reporte semanal

Descargar último reporte semanal

Inventario

Excel

Descargar Inventario

Ventas

Excel

Descargar Ventas

Alertas

Excel

Descargar Alertas

Ventas Hoy

Excel

Descargar Ventas Hoy

Ventas Semana

Excel

Descargar ventas Semana

Alertas Pendientes

Excel

Descargar Alertas Pendientes

Inventario Critico

Excel

Workflows - n8n

Inventario Pro

127.0.0.1:5000/reabastecimiento

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoría

Sistema de Gestión de Inventario

Reabastecimiento de inventario

Producto

Seleccione producto

Cantidad a agregar

Motivo

Compra a proveedor, ajuste, devolución...

Registrar reabastecimiento

Historial de reabastecimientos

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoria

Sistema de Gestión de Inventario

Historial de movimientos

Buscar SKU o detalle

Todos los tipos

Usuario

Filtrar

Limpiar

ID	FECHA	SKU	TIPO	CANTIDAD	DETALLE	USUARIO
85	2026-05-31 00:05:11.320123	HUEVOS-012	VENTA	15	Venta registrada desde la app	admin
84	2026-05-31 00:02:40.658107	ACEITE-009	VENTA	1	Venta registrada desde la app	ventas
83	2026-05-31 00:00:08.100518	AZUCAR-002	REABASTECIMIENTO	15	Devolución	admin
82	2026-05-30 23:58:59.282388	AZUCAR-002	VENTA	1	Venta registrada desde la app	admin
81	2026-05-30 23:58:48.913299	AZUCAR-002	VENTA	20	Venta registrada desde la app	admin

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoria

Sistema de Gestión de Inventario

Auditoría del sistema

Buscar evento o detalle

Buscar

Limpiar

ID	FECHA	EVENTO	DETALLE
76	2026-05-30 20:05:11.294000	STOCK ACTUALIZADO	Producto HUEVOS-012 actualizado. Vendido: 15, stock nuevo: 3. ALERTA: Bajo stock
75	2026-05-30 20:02:53.286000	REPORTE SEMANAL GENERADO	Reporte semanal generado correctamente en formato CSV con sugerencia de reorden
74	2026-05-30 20:02:40.629000	STOCK ACTUALIZADO	Producto ACEITE-009 actualizado. Vendido: 1, stock nuevo: 26
73	2026-05-31 00:00:08.100518	REABASTECIMIENTO	Se agregaron 15 unidades al producto AZUCAR-002. Motivo: Devolución
72	2026-05-30 19:58:59.218000	STOCK ACTUALIZADO	Producto AZUCAR-002 actualizado. Vendido: 1, stock nuevo: 0. ALERTA: Producto ya estaba en bajo stock, no se creó alerta duplicada

Workflows - n8n

Inventario Pro

127.0.0.1:5000/usuarios

Supermarket

Panel administrativo

Administrador | admin

Buscar módulo...

Dashboard

Productos

Ventas

Alertas

Reportes

Reabastecimiento

Historial movimientos

Auditoría

Sistema de Gestión de Inventario

1

Gestión de usuarios

+ Nuevo usuario

Buscar por nombre, usuari

Buscar

Limpiar

ID	NOMBRE	USUARIO	ROL	FECHA CREACIÓN	ACCIONES
6	Supervisor	supervisor	Supervisor	2026-05-29 17:20:33.823977	Editar Eliminar
5	Empleado Ventas	ventas	Ventas	2026-05-29 17:20:33.823977	Editar Eliminar
4	Encargado Inventario	inventario	Inventario	2026-05-29 17:20:33.823977	Editar Eliminar
2	Empleado Ventas	empleado	Empleado	2026-05-29 17:20:33.823977	Editar Eliminar
1	Administrador	admin	Admin	2026-05-29 17:20:33.823977	Editar Usuario actual

```

PS C:\Users\edinc\Documents\Cursos_UMG\Ingenieria de software\Proyecto Final inventario_n8n_Inventario_web\inventario-n8n> docker exec
-it inv_postgres psql -U n8n -d inventario
inventario=# \dt
inventario=# \d productos
               Table "public.productos"
   Column   |      Type      | Collation | Nullable |      Default
-----|-----|-----|-----|-----
 id         | integer        |           | not null | nextval('productos_id_seq'::regclass)
 sku        | character varying(50) |           | not null |
 nombre     | character varying(120) |           | not null |
 stock_actual | integer        |           | not null | 0
 stock_minimo | integer        |           | not null | 0
 updated_at | timestamp without time zone |           | not null | now()
Indexes:
    "productos_pkey" PRIMARY KEY, btree (id)
    "productos_sku_key" UNIQUE CONSTRAINT, btree (sku)

inventario=# 

```

```

PS C:\Users\edinc\Documents\Cursos_UMG\Ingenieria de software\Proyecto Final inventario_n8n_Inventario_web\inventario-n8n> docker exec -it inv_postgres psql -U n8n -d inventario
updated_at | timestamp without time zone | | not null | now()
Indexes:
    "productos_pkey" PRIMARY KEY, btree (id)
    "productos_sku_key" UNIQUE CONSTRAINT, btree (sku)

inventario=# \d auditoria
          Table "public.auditoria"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
id       | integer                |           | not null | nextval('auditoria_id_seq'::regclass)
fecha    | timestamp without time zone |           |          | now()
evento   | character varying(100) |           |          |
detalle  | text                   |           |          |
Indexes:
    "auditoria_pkey" PRIMARY KEY, btree (id)

inventario=#

```

```
inventario=# SELECT * FROM productos;
inventario=# \d ventas
```

Table "public.ventas"				
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('ventas_id_seq'::regclass)
fecha	timestamp without time zone		not null	now()
sku	character varying(50)		not null	
cantidad	integer		not null	
origen	character varying(20)		not null	'webhook'::character varying
raw_json	text			

Indexes:

"ventas_pkey" PRIMARY KEY, btree (id)

```
inventario=#
```

```
inventario=# \d alertas
```

Table "public.alertas"				
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('alertas_id_seq'::regclass)
fecha	timestamp without time zone		not null	now()
sku	character varying(50)		not null	
tipo	character varying(30)		not null	
mensaje	text		not null	
estado	character varying(20)		not null	'PENDIENTE'::character varying
prioridad	character varying(20)			'MEDIA'::character varying
fecha_resolucion	timestamp without time zone			
usuario_resolucion	character varying(100)			

Indexes:

"alertas_pkey" PRIMARY KEY, btree (id)

```
inventario=#
```

Conclusiones

El sistema desarrollado permitió automatizar el control de inventario de manera eficiente utilizando herramientas modernas como n8n, Flask y PostgreSQL.

La automatización redujo errores manuales, mejoró el control del stock y permitió generar alertas y reportes en tiempo real.

Además, el uso de Docker facilitó el despliegue local del sistema.